

MAT499 / 599

Project 3: Image Classification

April 17, 2026

DUE: 11:59pm on Wednesday April 29th, 2026

Submission: Please submit this project to canvas as an iPython Notebook (.ipynb file). The project is worth 25% and 20% of the overall grade for MAT499 and MAT599, respectively. While the submission is a python script, also focus on presenting the questions logically (write them out!), explaining what the code in each block does using comments, and answering the word questions fully in text blocks within the notebook.

Task: This project will attempt to model the Fashion-MNIST dataset (presented here: <https://arxiv.org/abs/1708.07747>). The data is comprised of 28 x 28 grayscale images of 70,000 fashion products from 10 categories, (7,000 images per category). The training set has 60,000 images and the test set has 10,000 images. Fashion-MNIST serves as a direct drop-in replacement for the original MNIST dataset for benchmarking machine learning algorithms, as it shares the same image size, data format and the structure of training and testing splits. I have made the data available on a google drive here: <https://drive.google.com/drive/u/0/folders/1-iLL9m2NfXp0PaSnRrJ-NaqBX4p5Cfz>.

Prior to running the following set a seed to 499 or 599 (your enrolled class) so that your results are the same for every run of your notebook. You will need to set the seed for the numpy.random, random, and torch libraries.

Question 1

Load the data from the csv files (both training and test) into a pandas dataframe. Create a validation set by selecting 3000 images of your training set at random. Check the distribution of the labels in the training and validation datasets.

Question 2

- (a) Treating the pixel values as independent predictors, train a fully connected network to predict the class of the image. Your architecture should have 2 hidden layers with 32 and 16 neurons, respectively, and use a ReLU activation function. Use the training and validation loss to guide the number of epochs you need to find a model that performs well. State the training loss, training accuracy, validation loss, and validation accuracy of the final trained model.

- (b) Assess the performance of the model on the test dataset. Generate a confusion matrix and from this calculate the overall accuracy and per-class accuracy. Which two classes are most frequently confused?

Question 3

- (a) Now let's treat the dataset as images. Reorganize the pixels to create grayscale images of size 28 x 28. Display one image from each class.
- (b) Train a CNN to predict the class of the input image. Your architecture should be:
- ⇒ Input layer
 - ⇒ Convolutional layer 1: 8 filters of size 3 x 3, stride = 1, padding = same, and ReLU activation function.
 - ⇒ Max Pooling layer: size of 2 x 2 and a stride of 2
 - ⇒ Convolutional layer 2: 16 filters of size 3 x 3, stride = 1, padding = same, and ReLU activation function.
 - ⇒ Max Pooling layer: size of 2 x 2 and a stride of 2
 - ⇒ A flattening function
 - ⇒ A fully connected layer with 32 neurons and ReLU activations
 - ⇒ Output layer

Use the training and validation loss to guide the number of epochs you need to find a model that performs well. State the training loss, training accuracy, validation loss, and validation accuracy of the final trained model.

- (c) Assess the performance of the model on the test dataset. Generate a confusion matrix and from this calculate the overall accuracy and per-class accuracy. Which two classes are most frequently confused? Can you hypothesize why this might be the case?

Question 4

- (a) These two models were chosen as they have approximately the same number of parameters. Verify this using code.
- (b) Under a controlled number of parameters, why would we expect the CNN outperform the Fully connected network for an image-based classification task?